

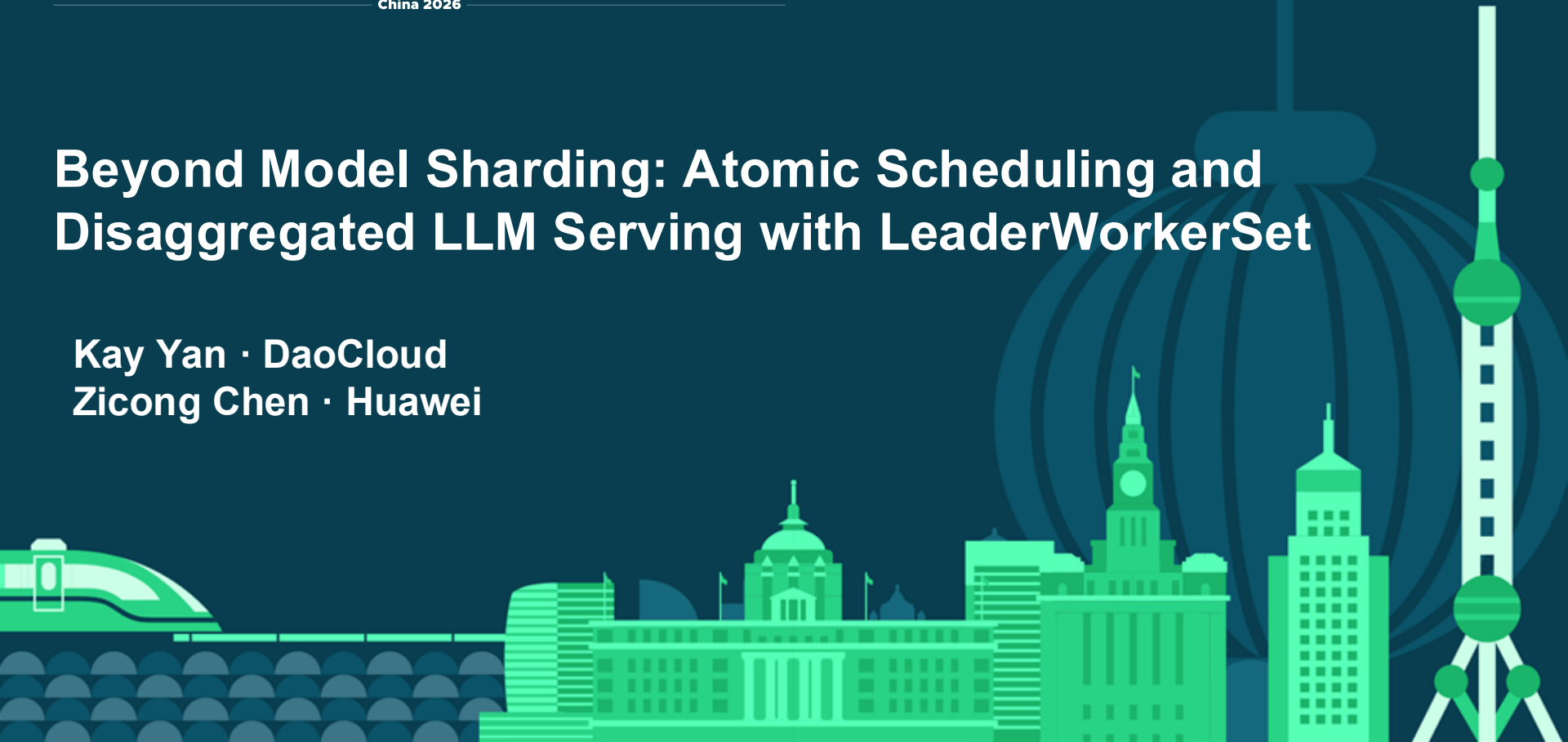


#KubeCon #CloudNativeCon
#OpenInfraSummit #PyTorchCon

China 2026

Beyond Model Sharding: Atomic Scheduling and Disaggregated LLM Serving with LeaderWorkerSet

Kay Yan · DaoCloud
Zicong Chen · Huawei



Speakers

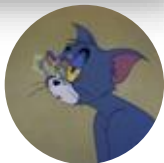


Zicong Chen

R&D Engineer

Huawei Cloud

Volcano Maintainer



Github: JesseStutler



Kay Yan

Principal Software Engineer

DaoCloud

LWS · Ilm-d Maintainer



Github: yankay

Agenda: Pods to distributed inference

1. Distributed inference: Beyond running Pods
2. LWS: A workload API for distributed inference
3. DisaggregatedSet: Orchestrating prefill and decode
4. Volcano: Gang scheduling
5. Roadmap and ecosystem: Production inference
6. Live demo and acknowledgments



KubeCon



CloudNativeCon



OpenInfra
SUMMIT > ASIA



PyTorch
CONFERENCE



China 2026

01

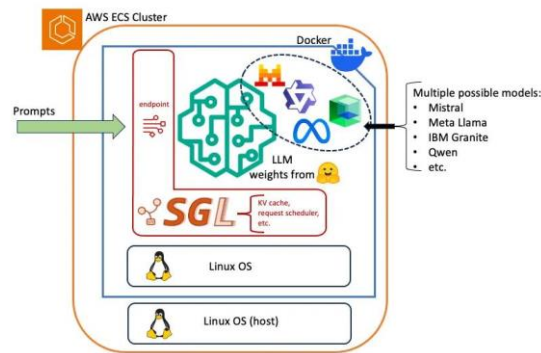
Distributed inference: Beyond running Pods



Model serving in one command

```
docker run --gpus all \
  --shm-size 32g \
  -p 30000:30000 \
  -v ~/.cache/huggingface:/root/.cache/huggingface \
  --env "HF_TOKEN=<secret>" \
  --ipc=host \
  lmsysorg/sglang:latest \
  python3 -m sglang.launch_server --model-path meta-llama/Llama-3.1-8B-Instruct --host 0.0.0.0 --port 30000
```

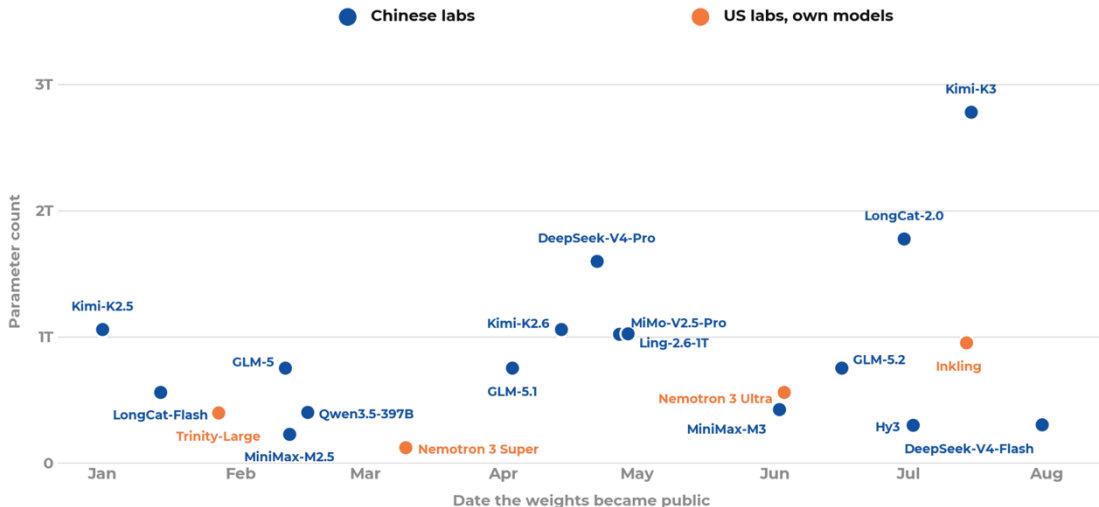
1. Run SGLang (or vLLM) in a Docker container
2. Download the Llama model from Hugging Face
3. Run inference on the GPU



Models outgrow a single node

The Frontier Ceiling Is Chinese

Selected flagship releases above 100B parameters, by the date the weights became publicly available



<https://huggingface.co/blog/state-of-open-models-summer-2026>

Model	Size
KIMI K3	2.8T
GLM 5.3	743B
DeepSeek V4 Pro	1.6T
GPT 6 Astra	~10T
Anthropic Mythos 5	~10T

~10T is a technical assumption

Model weights

> HBM / Node

One model service becomes a distributed system.



KubeCon



CloudNativeCon

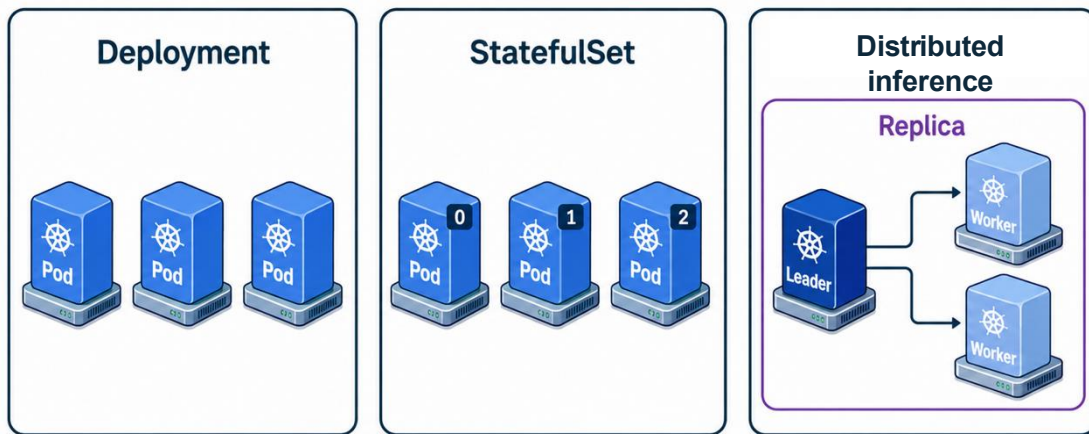


OpenInfra
SUMMIT > ASIA



PyTorch
CONFERENCE

Deployment / StatefulSet: What's missing?



Deployment

Replicates a single Pod template.

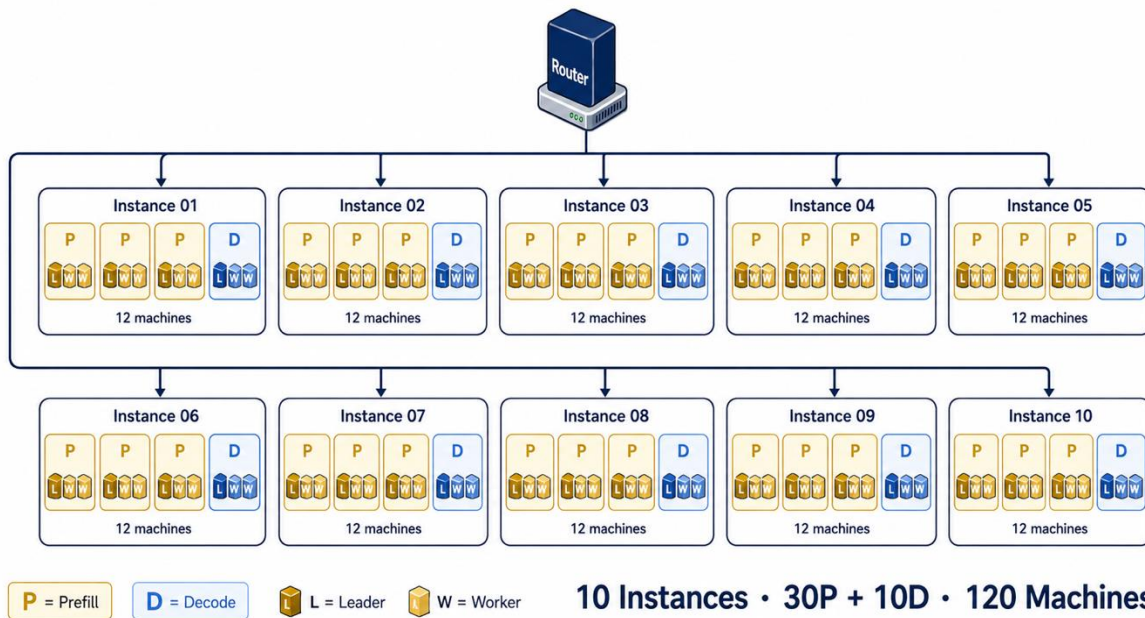
StatefulSet

Stable identities and ordering.

Distributed inference

One replica: a leader + workers.

Disaggregated inference: Prefill and decode



10 Instances • 30P + 10D • 120 Machines

Each group: 1 Leader + 2 Workers

Separate stages

Improves tokens/s/GPU

Prefill: Full prompt

Decode: Token by token

KV cache links the stages

StatefulSet manages Pods; group lifecycles and P/D coordination
still require custom orchestration.

Distributed inference: Operational challenges

Running Pods is only the first step.

- Gang scheduling: Can all resources be allocated together?
- Topology: Are group members in the right topology domain?
- Autoscaling: Can each role scale independently?
- Revision matching: Do updates keep prefill/decode aligned?
- Progressive rollouts: Will updates interrupt requests?



KubeCon



CloudNativeCon



OpenInfra
SUMMIT > ASIA



PyTorch
CONFERENCE

02

LWS: A workload API for distributed inference



LWS (LeaderWorkerSet)

An API for deploying a group of pods as a unit of replication.

Workload Model

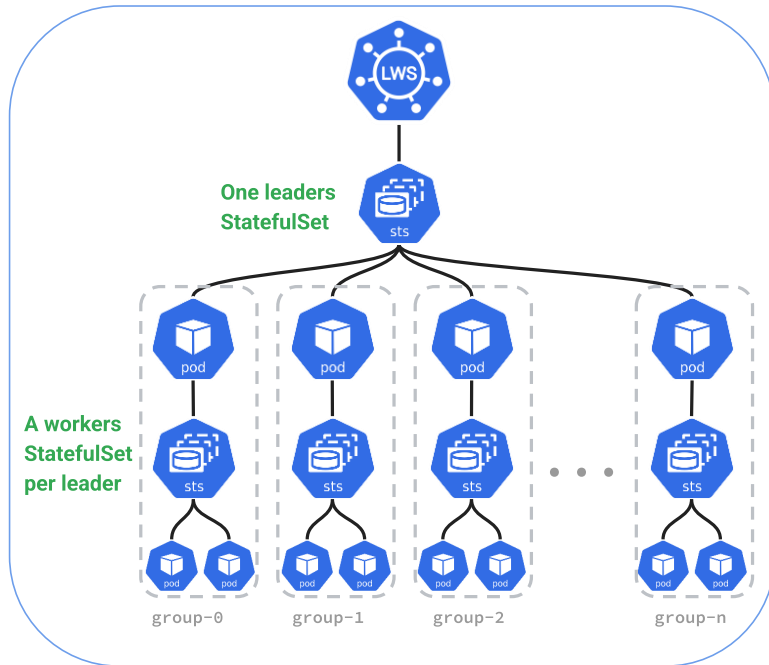
- A Pod group as one replica
- Unique Pod identity
- Parallel creation

Replica Management

- Scale subresource
- Rolling updates

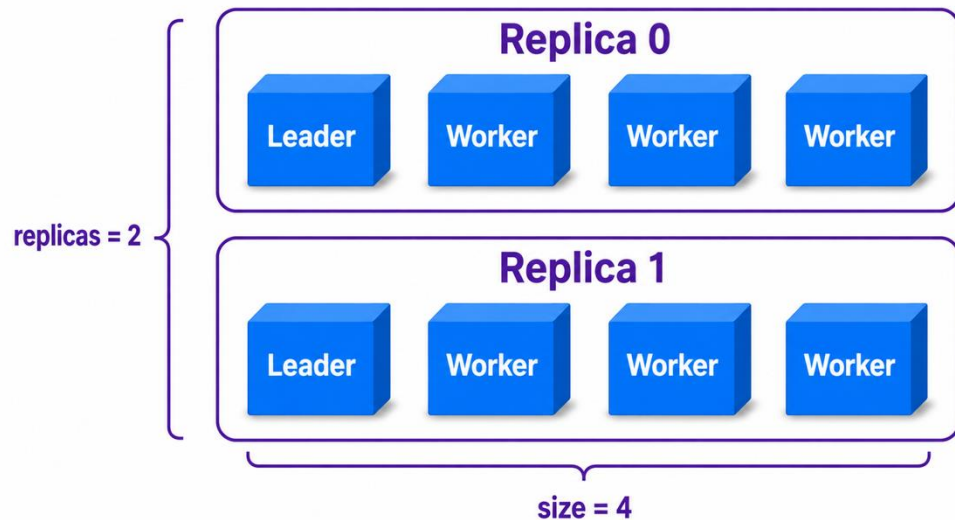
Scheduling & Resilience

- Gang Scheduling
- Topology-aware placement
- All-or-nothing restart



Defining a workload: Leader + workers

K8s-Native “Super Pod” Orchestration

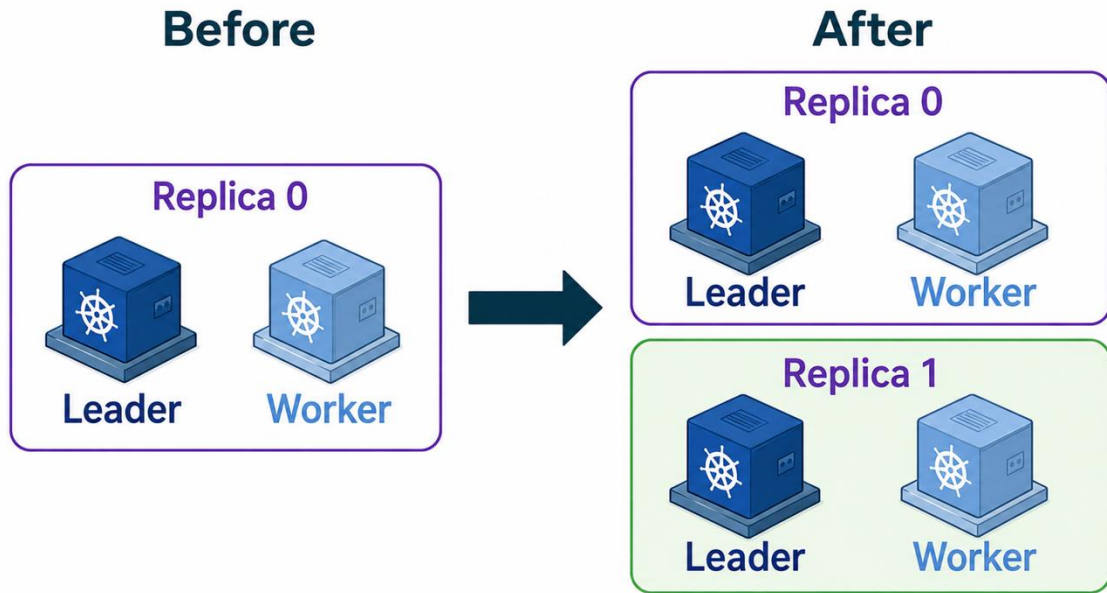


```
spec:
  replicas: 2
  startupPolicy: LeaderReady
  leaderWorkerTemplate:
    size: 4
    leaderTemplate:
      spec: ...
    workerTemplate:
      spec: ...
```

$\text{replicas: } 2 \times \text{size: } 4 = 8 \text{ Pods, including 2 leaders}$

leaderTemplate is optional;
if omitted, the leader uses workerTemplate.

Scaling out adds a complete replica



Example: 1 → 2 replicas

```
spec:  
  replicas: 1->2  
  startupPolicy: LeaderReady  
  leaderWorkerTemplate:  
    size: 2
```

Scale in whole replicas

In a 3P1D setup, each prefill replica is a complete group of leader and worker Pods.

Pod identity and leader discovery

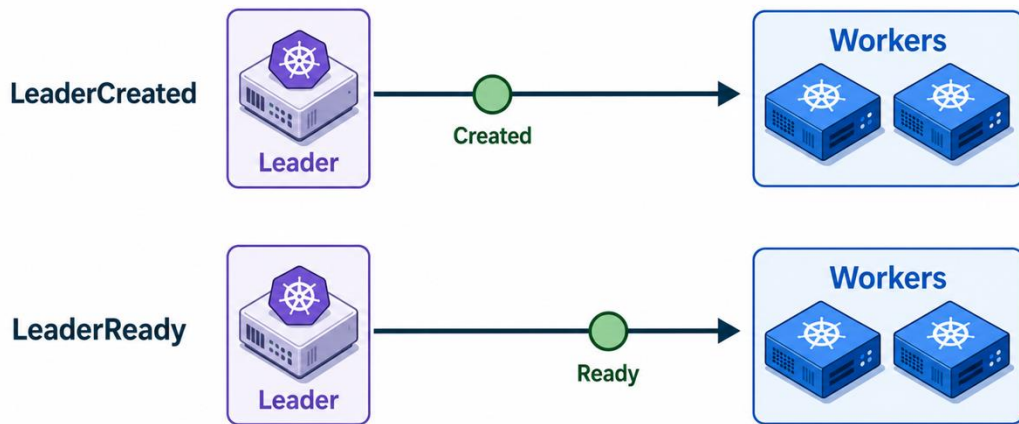
Stable identities connect Kubernetes objects to runtimes (vLLM, SGLang)

Environment Variable Injection

Key	Example
LWS_LEADER_ADDRESS	leaderworkerset-multi-template-0.leaderworkerset-multi-template.default
LWS_GROUP_SIZE	6
LWS_WORKER_INDEX	2



Startup policy: When to create workers



LeaderCreated

Default. The worker StatefulSet follows leader Pod creation.

LeaderReady

Create the worker StatefulSet once the leader Pod is Ready.

Restart policy: All-or-nothing recovery

```
spec:  
  leaderWorkerTemplate:  
    restartPolicy: RecreateGroupOnPodRestart
```

- **RecreateGroupOnPodRestart:** Recreate the group when a Pod fails. (*default*)



- **None:** Restart only the failed Pod; other Pods keep running.



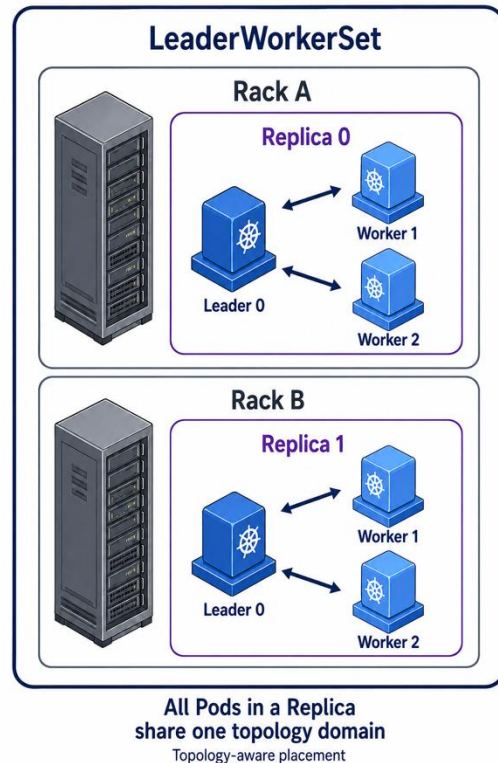
Topology-aware placement

Topology affects communication cost in distributed inference.

```
metadata:  
  annotations:  
    leaderworkerset.sigs.k8s.io/exclusive-topology:  
      topology.example.com/rack
```

Replica level: exclusive-topology maps a whole replica to an exclusive topology domain.

Subgroup level: Split a replica into smaller groups for Google TPU 3D topology.



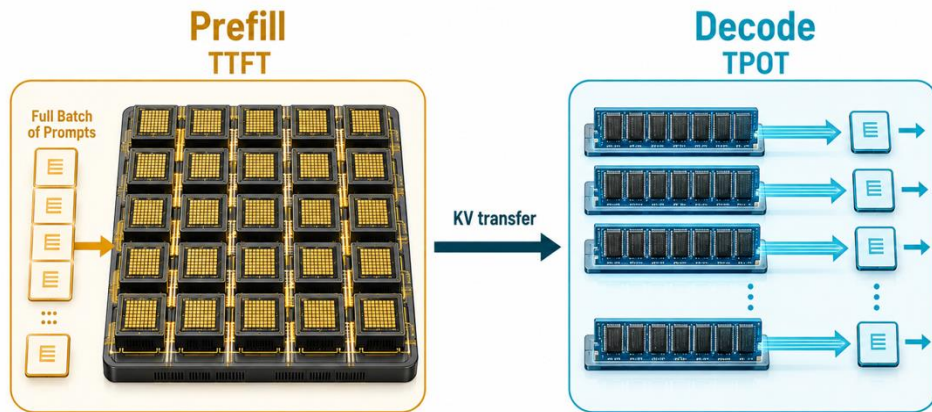
03

DisaggregatedSet: Orchestrating prefill and decode



Prefill and decode: Two distinct inference workloads

Coordinating prefill and decode poses a new challenge beyond the LWS leader-worker model.



Prefill

Typically compute-bound; mainly affects time to first token (TTFT).

Decode

Typically memory-bandwidth-bound; mainly affects time per output token (TPOT).

Previously: Two independent LWS workloads—functional, but without unified orchestration.

DisaggregatedSet

Building on LWS for advanced disaggregated multi-node inference.

Workload Model

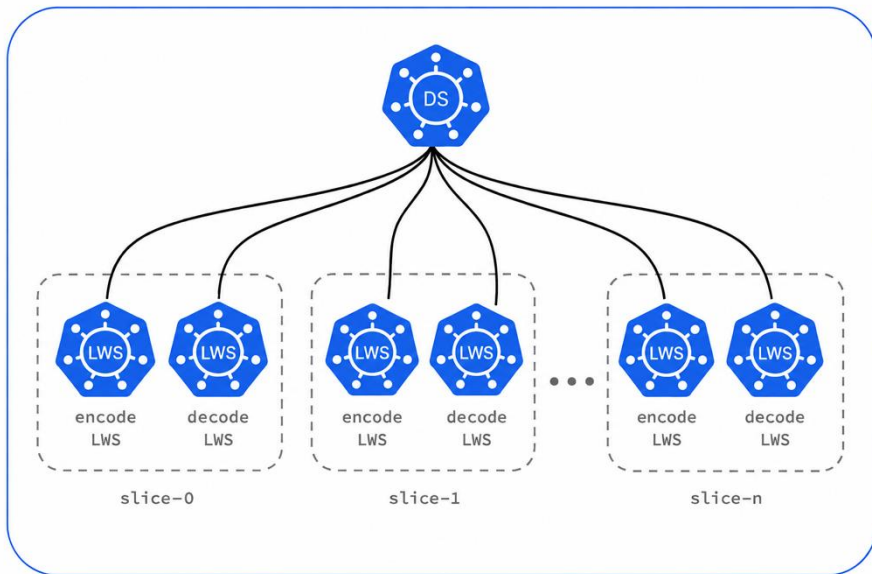
- Disaggregated Architecture Support
- Multi-role Workloads

Lifecycle Management

- Unified Lifecycle Management
- Coordinated N-Dimensional Rollouts
- Advanced Failure Handling

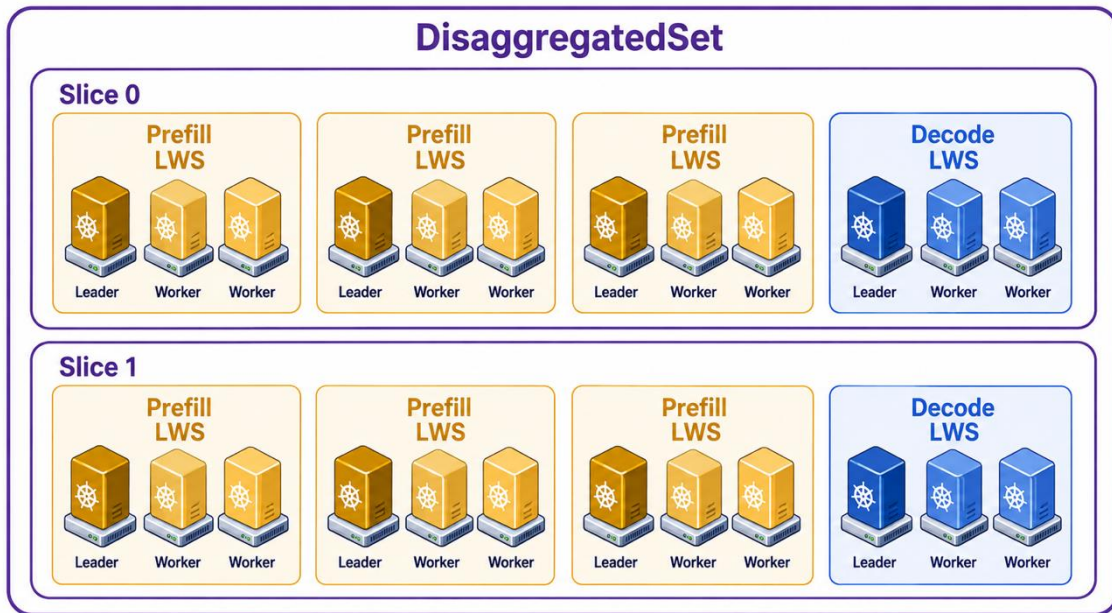
Service Orchestration

- Automatic Service Orchestration
- Revision-aware Routing



DisaggregatedSet: An orchestration layer above LWS

One service definition; multiple slices, each containing all roles

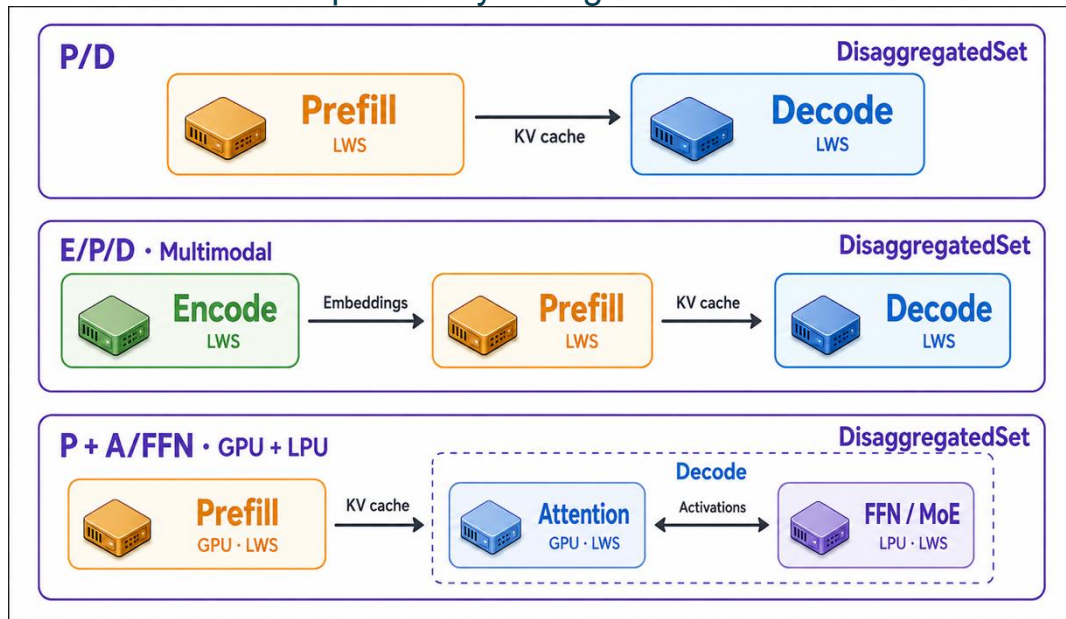


```
kind: DisaggregatedSet
spec:
  slices: 2
  roles:
    - name: prefill
      spec:
        replicas: 3
        leaderWorkerTemplate:
          size: 3
          leaderTemplate: ...
          workerTemplate: ...
    - name: decode
      spec:
        replicas: 1
        leaderWorkerTemplate:
          size: 3
          workerTemplate: ...
```

Avoid abbreviating DisaggregatedSet as DS: in Kubernetes, DS usually means DaemonSet.

Roles: More than prefill and decode

Each role is an independently configured LWS workload

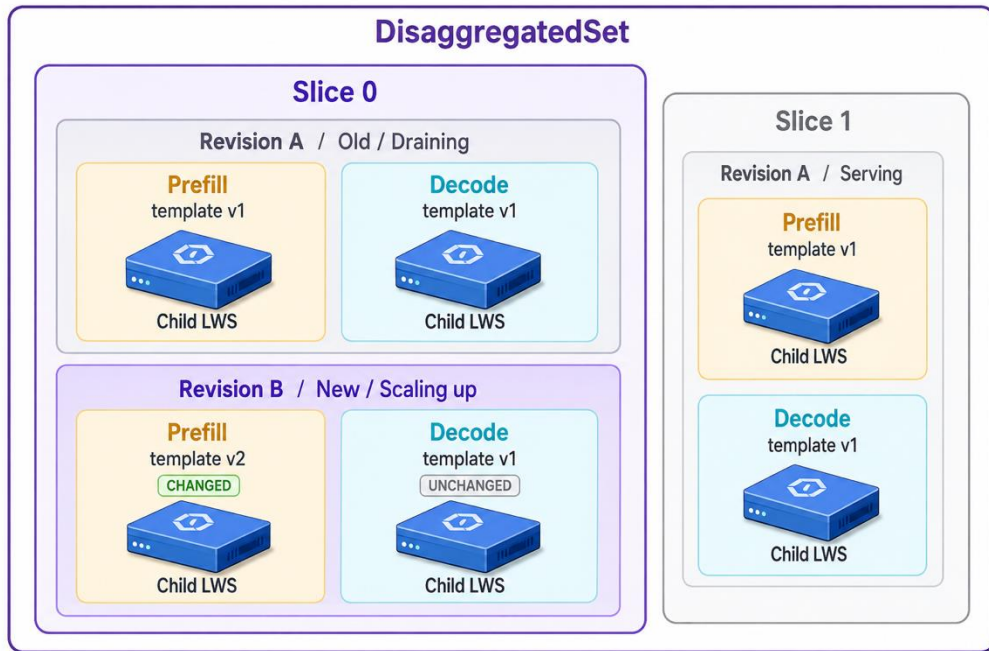


Configure each role independently

- 2–10 roles with custom names
- Replica count · replicas
- Group size · size
- Leader / worker Pod templates
- CPU / GPU / memory resources

A router is not a role: it is a separate traffic-routing layer, outside the model's compute topology.

Revisions: Old and new coexist without mismatches



One slice, multiple revisions

Revision A

Old combination scales down.

Revision B

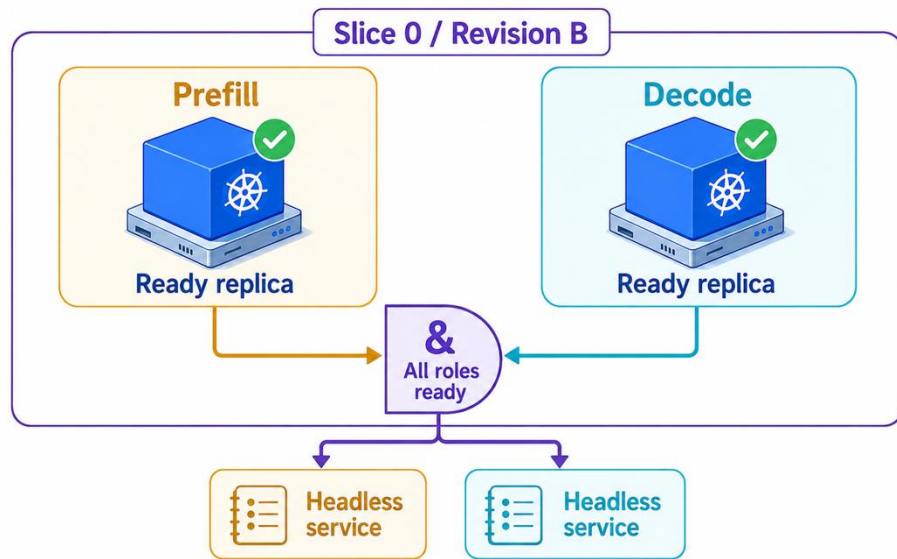
New combination scales up.

Changing only prefill also gives decode a new child LWS identity in Revision B, even with an unchanged template.

1 Child LWS Name
= Slice × Revision × Role

model-0-a1b2c3d4-prefill
| | |
Slice Revision Role

Versioned discovery: Services after all roles are Ready



```
apiVersion: v1
kind: Service
metadata:
  name: model-service-0-a1b2c3d4-prefill-prv
spec:
  clusterIP: None
  selector:
    disaggregatedset.x-k8s.io/name: model-service
    disaggregatedset.x-k8s.io/slice: "0"
    disaggregatedset.x-k8s.io/revision: a1b2c3d4
    disaggregatedset.x-k8s.io/role: prefill
```

Headless Services provide revision-isolated discovery, without request routing.

Coordinated rollouts: Per-slice revision handoff

All slices proceed concurrently, each independently by role

Slice	Stage	Revision A (old)	Revision B (new)	Action
0	Initial	2P / 1D	0P / 0D	Create LWS for all B roles
0	Grow new	2P / 1D	1P / 1D	Wait for all B roles to be Ready
0	Shrink old	1P / 1D	1P / 1D	Free capacity for the next step
0	Grow again	1P / 1D	2P / 1D	B reaches target capacity
0	Complete	0P / 0D	2P / 1D	Remove Revision A

maxSurge: 1 · maxUnavailable: 0

Four rules within each slice

Change one side

Grow new or shrink old per step

Wait for stability

Wait until all new roles are Ready

Bound capacity

$\text{old} + \text{new} \leq \text{target} + \text{maxSurge}$

Drain together

Leave no stranded roles in the old revision



KubeCon



CloudNativeCon



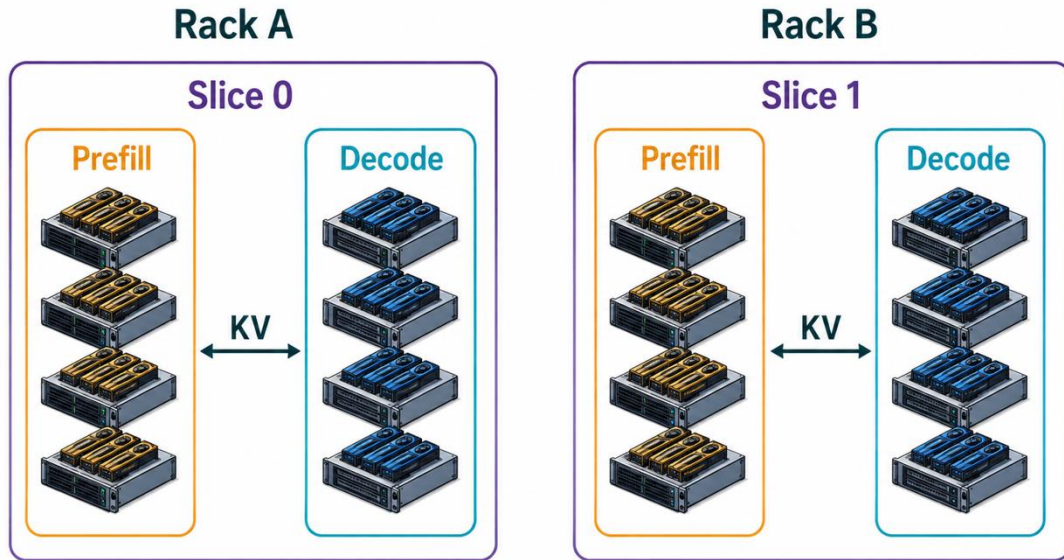
OpenInfra
SUMMIT > ASIA



PyTorch
CONFERENCE

PlacementPolicy: Co-locate each slice in a low-latency domain

Co-locate within a slice; spread across slices



```
kind: DisaggregatedSet
spec:
  slices: 2
  placementPolicy:
    type: ExclusiveSlice
    topology:
      topology.example.com/rack
  roles:
    - name: prefill
      spec: ...
    - name: decode
      spec: ...
```

Slices of one DisaggregatedSet must use distinct topology domains, so one failure domain cannot affect multiple slices.

Static / External: Who sets each role's replica count?

Scale prefill and decode independently to adjust the P/D capacity ratio.

Static

DisaggregatedSet
role.spec.replicas



Child LWS

```
- name: decode
  scaling:
    mode: Static
  spec:
    replicas: 2
```

External

HPA / KEDA



/scale
RoleScaler
[ds]-[role]



Child LWS

```
- name: prefill
  scaling:
    mode: External
  spec:
    leaderWorkerTemplate:
      size: 4
```

External mode currently requires slices: 1

DisaggregatedSetRoleScaler provides a stable target for independent autoscaling.
Slice count can be scaled; HPA / KEDA autoscaling remains on the roadmap.

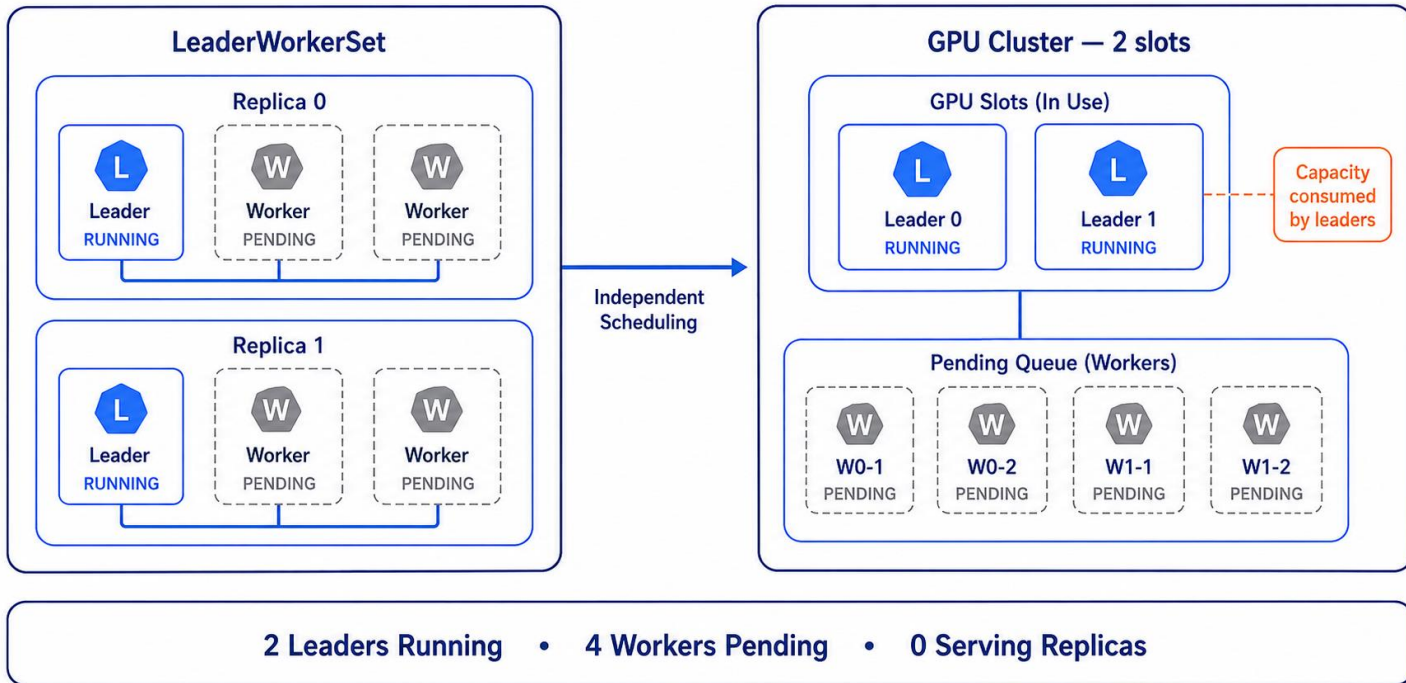
04

Volcano: Gang scheduling



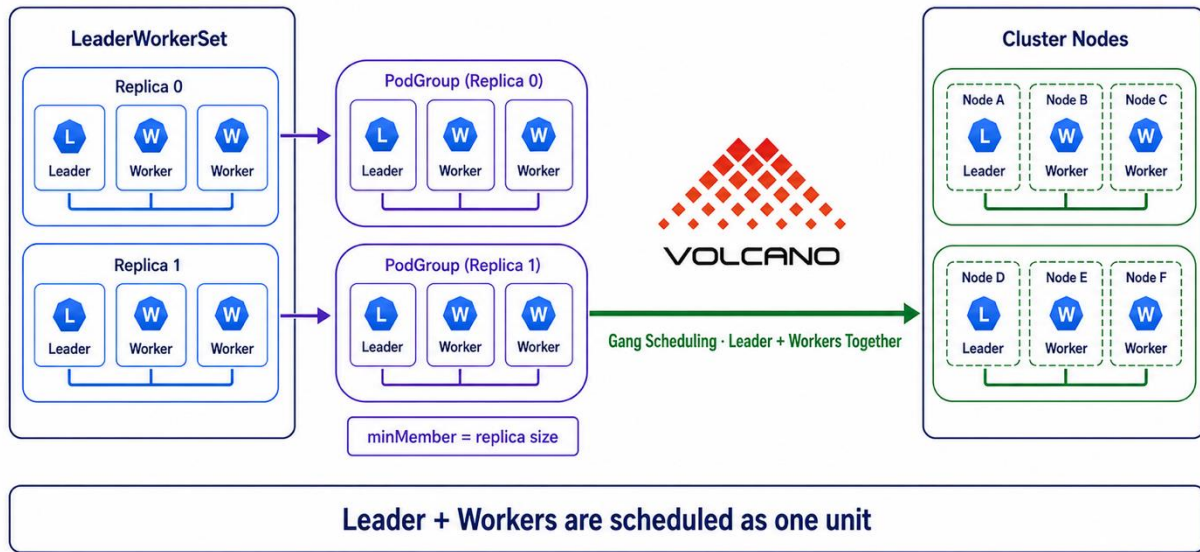
Why LeaderWorkerSet needs gang scheduling

Independent scheduling can fill every GPU slot with leaders while no replica becomes ready.



Volcano gang scheduling for LeaderWorkerSet

Each LWS replica becomes one scheduler-visible PodGroup.

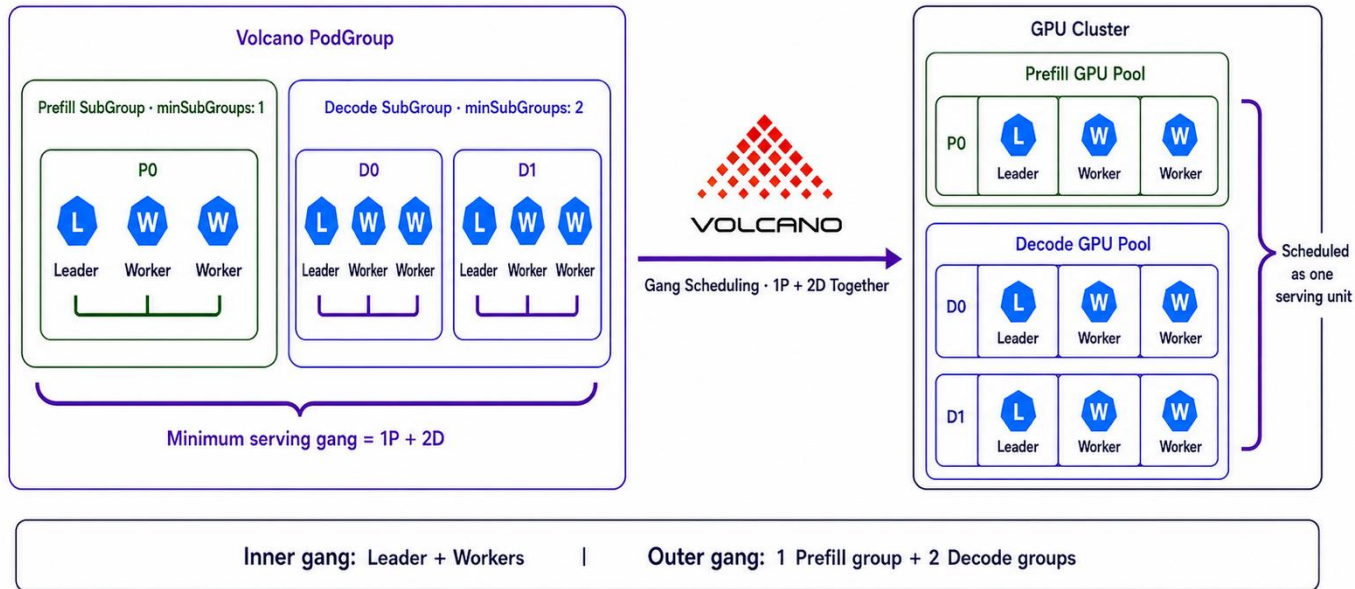


```
spec:  
  startupPolicy:  
    LeaderCreated  
  leaderWorkerTemplate:  
    size: 3  
    workerTemplate:  
      spec:  
        schedulerName:  
          volcano  
  
# generated PodGroup  
spec:  
  minMember: 3
```

size: 3 → minMember: 3 → schedule Leader + Workers as one gang

Hierarchical gang scheduling for DisaggregatedSet

Future mapping: keep each LWS replica gang, then schedule a minimum 1P + 2D serving slice.



podgroup.yaml

```
spec:  
  minMember: 9  
  subGroupPolicy:  
    - name: prefill  
      subGroupSize: 3  
      minSubGroups: 1  
    - name: decode  
      subGroupSize: 3  
      minSubGroups: 2
```

Future Direction



KubeCon



CloudNativeCon



OpenInfra
SUMMIT > ASIA



PyTorch
CONFERENCE

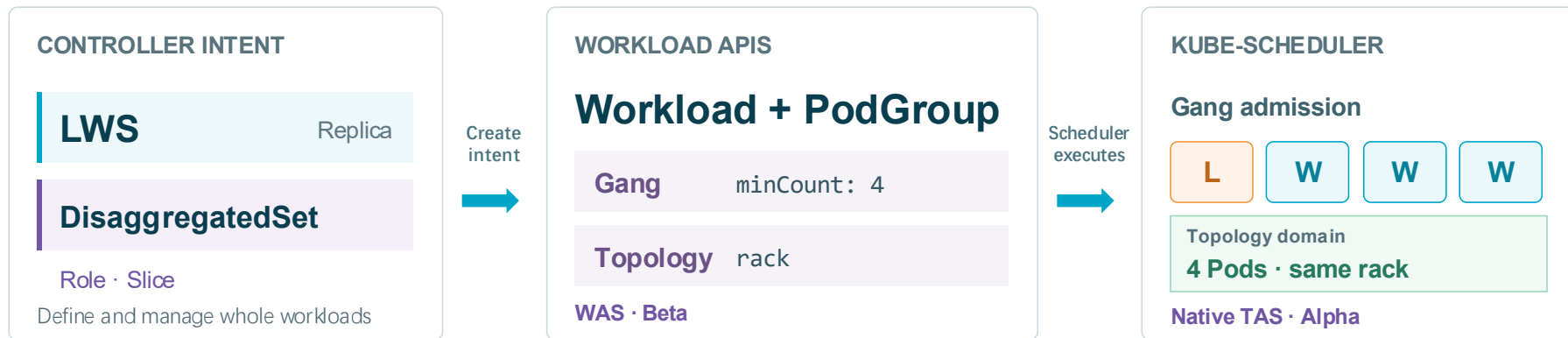
05

Roadmap & ecosystem: Toward production inference



Roadmap: Native Kubernetes scheduling

WAS · Workload-Aware Scheduling | TAS · Topology-Aware Scheduling



```
kind: Workload
spec:
  podGroupTemplates:
  - name: replica
    schedulingPolicy:
      gang: { minCount: 4 }
    schedulingConstraints:
      topology:
        - key: topology.kubernetes.io/rack
```

Upstream and integration status

Upstream API

WAS Beta · TAS Alpha

LWS / DisaggregatedSet integration

Roadmap



KubeCon



CloudNativeCon



OpenInfra
SUMMIT > ASIA



PyTorch
CONFERENCE

Volcano: Gang scheduling support

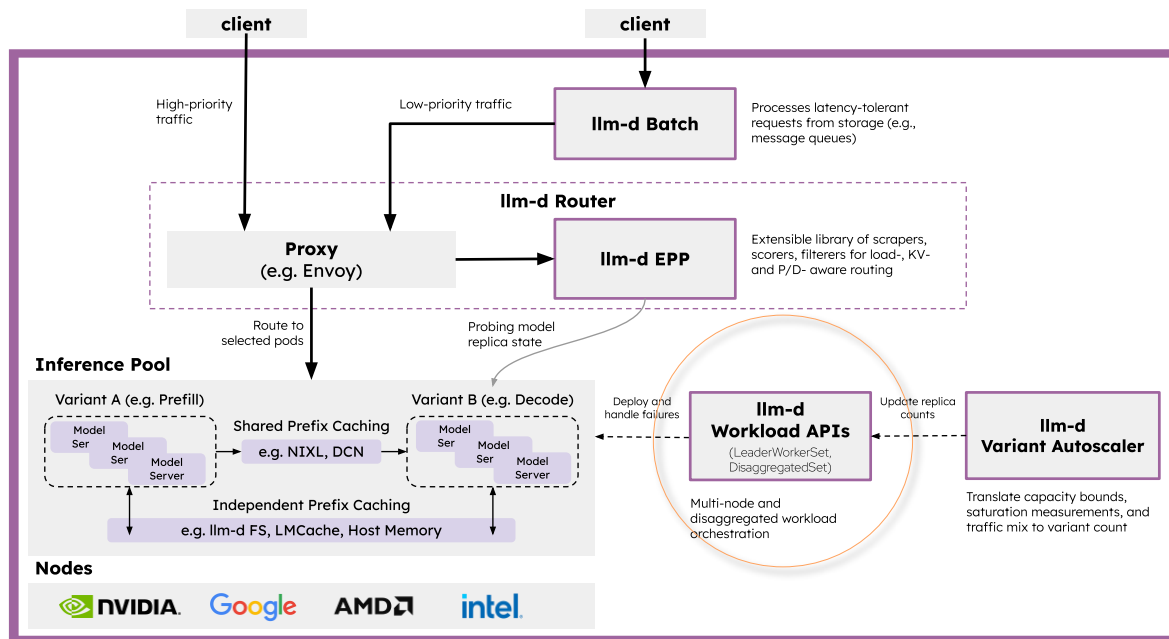
The scheduler provider interface connects LWS APIs to a broader scheduling ecosystem.



The integration model leaves room for additional scheduler providers.

llm-d: LWS in production inference

In llm-d, LWS orchestrates workloads and manages the full lifecycle of inference instances.



Routing
Router / EPP

Multi-node inference
LeaderWorkerSet

P/D orchestration
DisaggregatedSet (new)

Inference engines
vLLM, SGLang

Deployment guides

- [P/D Disaggregation: Separate prefill / decode deployment](#)
- [P/D with DisaggregatedSet: Unified prefill / decode orchestration](#)



KubeCon



CloudNativeCon



OpenInfra
SUMMIT > ASIA



PyTorch
CONFERENCE

Dynamo: Multi-node inference with LWS

Dynamo Operator: Grove / LWS paths; DisaggregatedSet integration in progress

Service definition

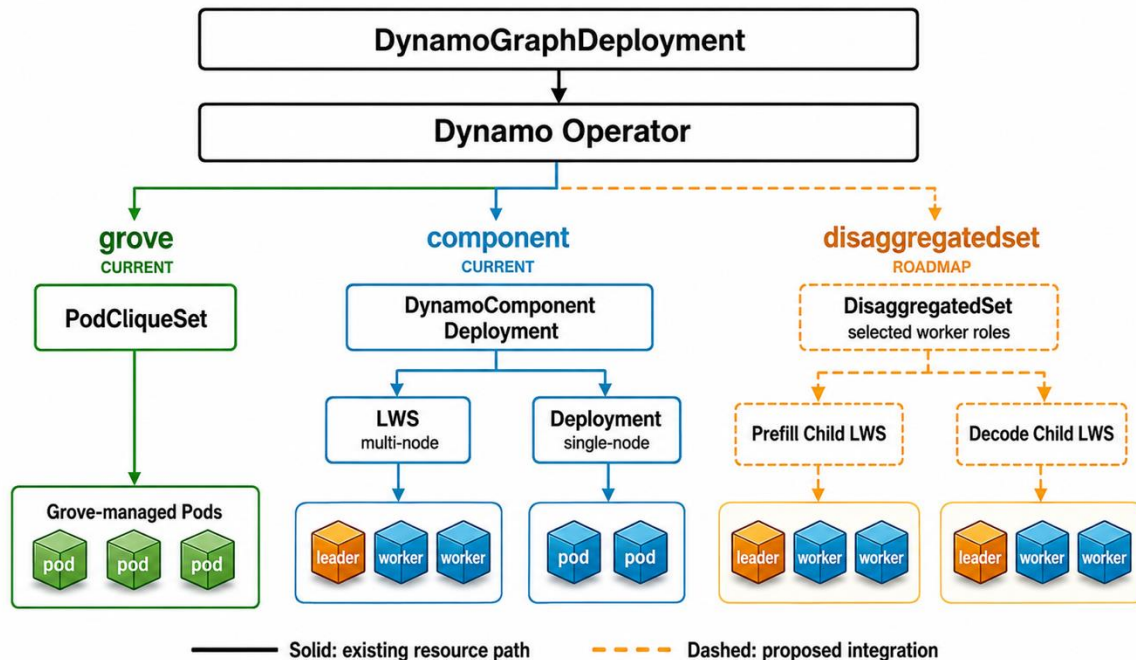
DynamoGraphDeployment
Defines the full inference graph.

Resource orchestration

Custom CRD → LWS

Integrations

Supports LWS and Grove



Shared workload APIs for multi-node inference

Adoption, compatibility and integration vary; shared APIs span platforms and inference stacks.

LWS

Whole replica

+

DisaggregatedSet

Multi-role coordination

From multi-node replicas
to complete inference services



Google

GKE multi-node



Red Hat

OpenShift



DaoCloud

Token Factory



Volcano

LWS Gang
Scheduling



AWS

EKS deployment



Alibaba Cloud

ACK deployment



vLLM

Deployment path



SGLang

Deployment path



llm-d

LWS /
DisaggregatedSet



Dynamo

Operator integration



KServe

LLMInferenceService



NIM

NVIDIA deployment



OME



Kthena

LWS API compatible



llmaz



JetStream

GKE / TPU
deployment



RBG

LWS compatible



Kubeflow

Trainer

LWS adoption and compatibility; not all projects integrate DisaggregatedSet. Projects shown in no particular order.



KubeCon



CloudNativeCon



OpenInfra
SUMMIT > ASIA



PyTorch
CONFERENCE

China 2026

<https://ws.sigs.k8s.io/docs/adoption/>

06

Live demo and acknowledgments

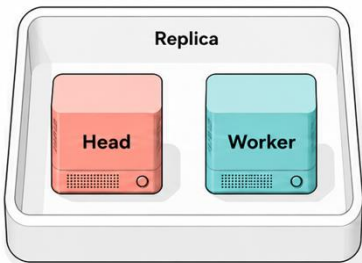


Demo: From simulation to real Qwen P/D inference

01 LWS

One replica, two Pods

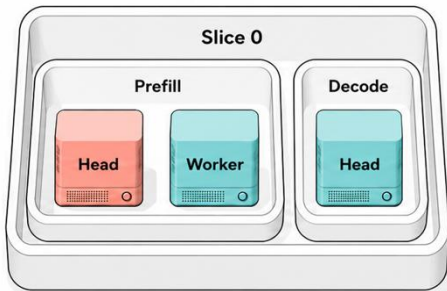
Simulated deployment



02 DisaggregatedSet

One service, two roles

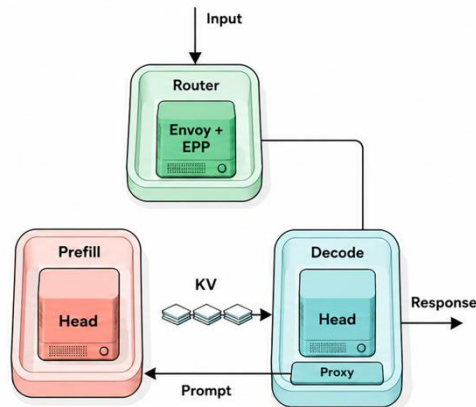
Simulated deployment



03 Ilm-d

Qwen, P/D disaggregation

Real deployment



Contribute: Shape what comes next for LWS

A real-world problem can spark the next improvement

01 Share use cases

Issue

Multi-node inference, P/D orchestration, reproducible issues

02 Join the design

KEP

For example: WAS / TAS integration and multi-role workloads

03 Submit improvements

PR

Examples, tests, documentation or code



[kubernetes-sigs/lws](https://github.com/kubernetes-sigs/lws)

lws.sigs.k8s.io

Kubernetes Slack
#sig-apps



KubeCon



CloudNativeCon



OpenInfra
SUMMIT > ASIA



PyTorch
CONFERENCE

China 2026



China 2026

Thank you

Q&A

